SecureVote

Your Vote, Your Voice, Secure.

Secure Voting Cryptography

As digital transformation reshapes democracy, secure cryptographic protocols are essential for protecting vote integrity. This project presents an electronic voting system using RSA, unique N1N_1N1 and N2N_2N2 codes, and blind signatures to ensure the secure distribution of responsibilities among the commissioner, administrator, anonymizer, and counter, which guarantees voter anonymity, secure authentication, and tamper-proof election results

TEAM MEMBERS

M1
BENTALEB
LISA

Blind signature

M2
DIFFALLAH
IMENE

RSA&CORE

M3
LAMRIA
Mahmoud
Noureddine

REST API & Wiring

M4
ARIANE
DJAD

REST API & Wiring

M5
BOUSSEKINE
MOHAMED ISMAIL

Frontend & Integration

Outline -- Structure & Speakers

01 **Global Architecture**

ARIANE DJAD

02 **RSA Cryptography**

DIFFALLAH Imene

03 **Blind Signatures**

BENTALEB Lisa

06 **TTh and Hashing functions**

Noureddine

05 **Problem: Credential Distribution**

Ismail

06 **Practical Project Overview**

All

GLOBAL ARCHITECTURE

Commissioner
port:5001

Anonymiser
port:5003

Administrator
port:5002

Counter
port:5004

5 Independent Services each can run on its own machine

RSA &Code Snippets

RSA Cryptosystem

Generation	$N = p \times q,\ \varphi(N) = (p-1)(q-1)$
Public key	$e = 65537,\ \text{pgcd}(e, \varphi(N)) = 1$
Private key	$d \equiv e^{-1} \mod \varphi(N)$
Encrypt	$c = m^e \mod N$
Decrypt	$m = c^d \mod N$
Sign	$s = m^d \mod N$
Verify	$s^e \equiv m \pmod N$ ✓

RSA CORE — crypto/rsa_core.py

`is_prime` `(n, rounds=20) → bool`	Tests if a number is prime using **Miller-Rabin** with 20 random witnesses. False-positive rate < 4^{-20}.
`generate_prime` `(bits=1024) → int`	Generates a random **odd number** of exactly `bits` bits and loops until **is_prime** passes.
`extended_gcd` `(a, b) → tuple`	**Extended Euclidean Algorithm** — returns (gcd, x, y) such that ax + by = gcd. Foundation for mod_inverse.
`mod_inverse` `(a, m) → int`	Computes $\mathbf{a^{-1} \bmod m}$ using extended_gcd. Used to derive the private key d from e and φ(N).
`generate_rsa_keypair` `(bits=2048) → tuple`	Generates two primes p, q. Computes **N = p×q**, verifies gcd(e, φ(N))=1, returns **(e, d, N)**.

Blind signature & code snippets

```python
def blind_message(m: int, e: int, N: int) -> tuple[int, int]:
    if not (0 < m < N):
        raise ValueError(f"m must satisfy 0 < m < N. Got m={m}.")

    # Cryptographically secure blinding factor
    while True:
        k = secrets.randbelow(N)
        if k > 1 and math.gcd(k, N) == 1:
            break

    r = pow(k, e, N)
    m_masked = (m * r) % N
    return m_masked, k


def sign_blind(m_masked: int, d: int, N: int) -> int:
    if not (0 <= m_masked < N):
        raise ValueError(f"m_masked must be in [0, N).")
    return pow(m_masked, d, N)


def unblind_signature(s_blind: int, k: int, N: int) -> int:
    if math.gcd(k, N) != 1:
        raise ValueError("k must be coprime with N.")
    k_inv = pow(k, -1, N)
    return (s_blind * k_inv) % N
```

Blind Signature — Full Protocol

VOTER **Blinds the ballot**

$$m' = m \cdot k^e \mod N_a$$

k random, coprime with N_a. Admin only sees m'.

ADMIN **Signs blindly**

$$m'' = (m')^d \mod N_a$$

Administrator authorises without seeing vote content.

VOTER **Unblinds**

$$s = m'' \cdot k^{-1} \mod N_a$$

Valid RSA signature on original m — k cancels out.

COUNTER **Verifies**

$$s^e \equiv m \pmod{N_a} \checkmark$$

RSA is multiplicatively homomorphic: $(m \cdot k^e)^d = m^d \cdot k$.

TTh and Hashing functions , N1 and N2

TTH (Toy Tetragraph Hash)

Splits input into 4-character blocks, applies iterative accumulation + diffusion mod 26. Output: 4 uppercase letters.

```
TTH(AF15GH258ZQP) = ACSQ
```

N1

Unique access code distributed by email. Marked used after voting. $36^{12} \approx 10^{18}$ combinations.

N2

hashed(N2) stored, raw N2 destroyed. Never sent by email.

More Secure Hash Variants

The initial TTH specification is a simplified educational model: it splits the input into 4-character blocks, applies iterative accumulation and diffusion modulo 26, and produces 4 uppercase letters.
Our implementation extends this idea into a stronger 8-character hexadecimal TTH using:

- 16-value blocks
- modulo-256 arithmetic
- XOR-based mixing
- hexadecimal output

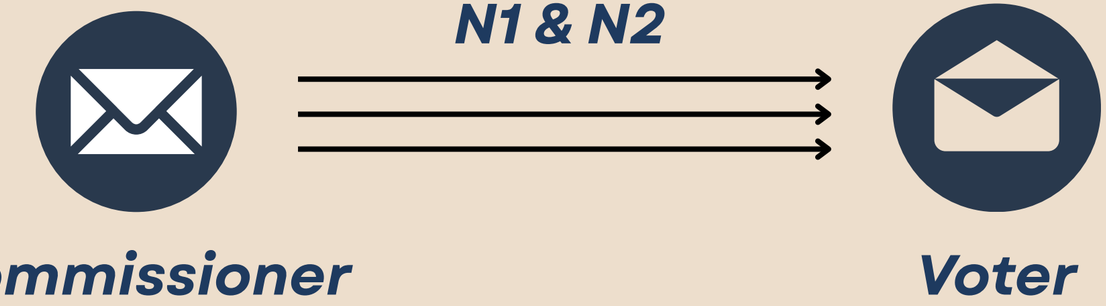
For stronger real-world security, the system also supports SHA-256, a standard cryptographic hash function widely used in secure systems.
Finally, the project provides a hash_n2() function that allows choosing between the pedagogic TTH mode and the secure SHA-256 mode.

Who Can See Which Parts?

Entity	Private key	what can it see
Commissioner	❌	N1's and tth(N2) Never sees any ballot or vote content
Administrator	✅ d_a, N_a	❌ Sees m′ only the blinded vote and N1
Anonymiser	❌	❌ Encrypted so nothing , N1 only
Counter	✅ d_D, N_D	✅ After decryption in the end of session: N2 , vote. but never N1
Voter	❌	Own vote only and own N1 , N2 only
Website	❌	❌ Relay only

HOW TO SECURLY DELIVER N1 AND N2 CODES ?

To securely distribute N1 and N2, the Commissioner sends both codes directly to the voter's email address immediately after generation. The plaintext N2 is then permanently discarded and never stored anywhere in the system. Only the hashed value TTH(N2) is retained by the Commissioner, ensuring that only the voter possesses the original N2 authentication token.



Conclusion

Voter anonymity

Blind signatures ensure the administrator authorises votes without ever learning their content.

Secure authentication

N1 and hashed N2 codes guarantee each voter is legitimate and can vote only once.

Tamper-proof results

RSA signatures on each ballot let the counter verify integrity without needing the voter's identity.

Distributed trust

Five independent services — no single entity holds enough information to compromise the election.

Project DEMO

THANK YOU!

SecureVote

Your Vote, Your Voice, Secure.